

Recuperación

Log y checkpoints · UNDO/REDO · actualización diferida e inmediata · paginación de sombra · backups

Bases de Datos (Relacionales) · Transacciones, Concurrency y Recuperación (parte 2)

Ing. Informática · Lic. en Cs. de Datos

Ricardo Di Pasquale
Alejandro Isidro
Edgardo Sánchez
Delfina Las Heras

AGENDA

1. ¿Por qué recuperación? Tipos de falla
2. El log de transacciones y el checkpoint
3. Proceso de recuperación: log vs. backups
4. Actualización diferida (REDO)
5. Actualización inmediata (UNDO/REDO, WAL)
6. Paginación de sombra
7. Recuperación catastrófica (backups)

MOTIVACIÓN

¿Por qué recuperación?

Recuperación: garantizar ACID ante fallas

El DBMS necesita un mecanismo de recuperación para sostener **Atomicidad** y **Durabilidad** cuando algo sale mal. Las fallas se agrupan en tres categorías:

Categoría	Ejemplos
De transacción / lógica	rollback de la aplicación o del DBMS; el Transaction Manager aborta por deadlock
De sistema	falla de energía o de memoria; error del sistema operativo
De medios (catastrófica)	falla de disco; problemas y catástrofes físicas

Las dos primeras se resuelven con el log; la tercera exige backups físicos.

El log (bitácora) de transacciones

Le sigue la pista a todas las operaciones para poder deshacerlas o rehacerlas. Tipos de entrada:

Entrada del log	Qué registra
Inicio de transacción	el comienzo de T
Escritura de recurso	el dato modificado (valor viejo / nuevo)
Fin de transacción	commit (éxito) o abort / rollback
Checkpoint	hasta dónde se volcaron las entradas a los archivos físicos

Se mantiene en disco y conviene respaldarlo para protegerlo de fallas catastróficas.

Checkpoint y proceso de recuperación

El **checkpoint** marca hasta qué punto las entradas del log ya se volcaron a los archivos físicos de datos. Según la falla, hay dos caminos:

Falla común (transacción en curso): alcanza con el log → deshacer y/o rehacer lo que estaba a medio hacer.

Falla catastrófica (medios): recurrir a los backups físicos (totales e incrementales) + el log.

BASADAS EN EL LOG

Actualización diferida e inmediata

Actualización diferida (deferred update)

No se graban los datos al almacenamiento físico durante la transacción: se escribe al log al confirmar, y recién en el checkpoint se vuelca a los archivos físicos.

- ▶ **Premisa 1:** una transacción no modifica la base antes de su punto de commit.
- ▶ **Premisa 2:** no llega al commit sin antes asentar todas sus escrituras al log y forzar su flush a disco.

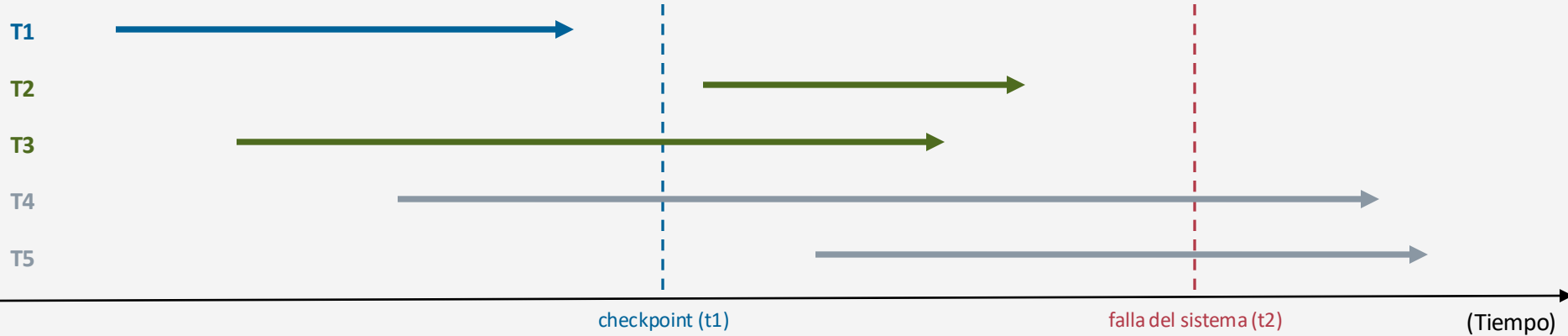
Si una transacción falla antes del commit: no hay nada que deshacer. La recuperación es sólo REDO.

Diferida · procedimiento de recuperación

- ▶ El DBMS arma dos listas: transacciones confirmadas desde el último checkpoint, y transacciones activas (Ta).
- ▶ REDO: rehacer, desde el log, todas las escrituras de las confirmadas, en el orden en que aparecen.
- ▶ Las activas (no confirmadas) se cancelan y deberán reintroducirse.

Optimización: procesar de atrás hacia adelante y rehacer cada recurso una sola vez (su última escritura).

Diferida · ejemplo



T1: ya en disco → nada.

T2, T3: confirmadas tras el checkpoint pero no volcadas → REDO.

T4, T5: sin commit → ignorar.

Actualización inmediata (immediate update)

Los datos se graban directo al almacenamiento físico. Para poder recuperar, se usa **escritura anticipada en el log**: el log se escribe antes que los datos.

- ▶ Dos listas: confirmadas desde el último checkpoint (T_c) y activas (T_a).
- ▶ UNDO: deshacer todas las operaciones de las activas, en el orden opuesto al que se asentaron.
- ▶ REDO: rehacer las operaciones de las confirmadas, en el orden en que se escribieron.

A diferencia de la diferida, acá SÍ hay que deshacer (UNDO) lo escrito por transacciones no confirmadas.

Diferida vs. inmediata

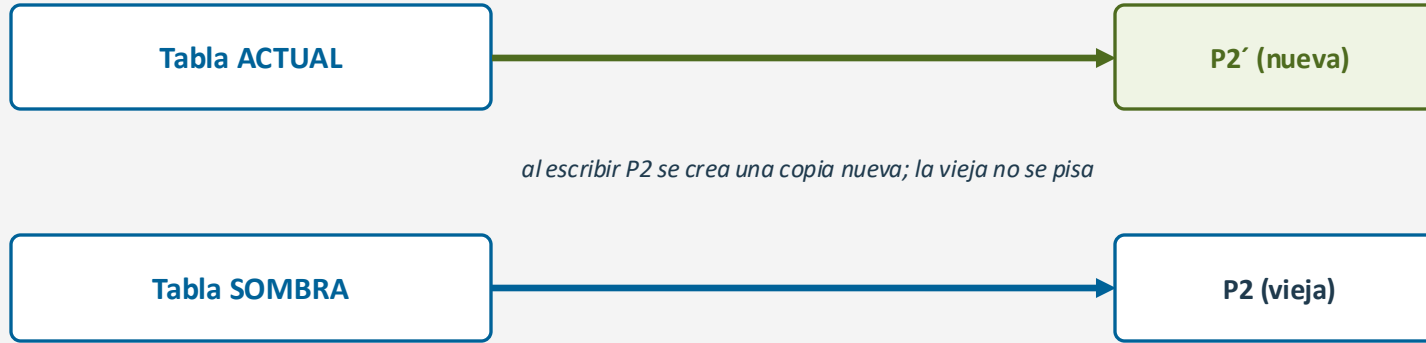
Aspecto	Diferida	Inmediata
¿Cuándo se graba a disco?	recién al confirmar (checkpoint)	de inmediato, durante la transacción
Uso del log	escribe y hace flush antes del commit	escritura anticipada (WAL): log antes que datos
Si falla antes del commit	no hay nada que deshacer	hay que deshacer (UNDO) lo escrito
Recuperación	sólo REDO (confirmadas)	UNDO (activas) + REDO (confirmadas)

En ambas, los checkpoints acotan cuánto log hay que reprocesar.

SIN DESHACER NI REHACER

Paginación de sombra

Paginación de sombra (shadow paging)



- ▶ Al iniciar la transacción, la tabla de páginas actual se copia a una tabla sombra (en disco).
- ▶ Falla → se descarta la tabla actual y se vuelve a la sombra (recuperación trivial).
- ▶ Commit → se descarta la sombra y se liberan las páginas viejas.

ANTE LO PEOR

Recuperación catastrófica

Esquema de backups



Log transaccional (continuo)

Restaurar = último Full + su último diferencial + el log (roll-forward hasta el instante de la falla).

Síntesis

- ▶ El log y los checkpoints son la base de la recuperación.
- ▶ Diferida: no toca el disco hasta el commit → recuperación sólo con REDO.
- ▶ Inmediata: escribe ya al disco → recuperación con UNDO + REDO.
- ▶ Paginación de sombra: copia-en-escritura; recuperar = volver a la tabla sombra.
- ▶ El checkpoint acota cuánto log hay que reprocesar tras una falla.
- ▶ Falla de medios (catastrófica): backups full + diferenciales + log.

¡Muchas gracias! Q&A

Seguinos



@ingeagrariasuca



@IngeagrariasUCA



Ingeniería y Ciencias Agrarias

Contacto



Av. Alicia Moreau de Justo 1500
C1107AAZ, Buenos Aires, Argentina



0810-220-0822



inge_agrarias@uca.edu.ar